VIT-AP UNIVERSITY

Experiment 10: Development of Naive Bayes Classifier for Classification of Financial Risk

CSE-3008 Introduction to Machine Learning

24BCA7027
Shaik Barakh Chishti
CSE-AIML

1 Aim

To develop and implement a Naive Bayes classifier to predict financial risk (loan default) using the Financial Risk Classification Dataset, and to evaluate its performance using 4-fold cross-validation and various statistical visualizations.

2 Implementation

2.1 Algorithm: Naive Bayes Classifier

Naive Bayes is a probabilistic classifier based on Bayes' Theorem with an assumption of independence among predictors. For continuous data, the Gaussian Naive Bayes model is used:

$$P(x_i|y) = \frac{1}{\sqrt{2\pi\sigma_y^2}} \exp\left(-\frac{(x_i - \mu_y)^2}{2\sigma_y^2}\right) \tag{1}$$

The model calculates the posterior probability for each class and selects the one with the highest probability.

2.2 Python Code Implementation

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import KFold
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, precision_score,
   recall_score, f1_score, mean_squared_error, r2_score, \
    confusion_matrix
from sklearn.preprocessing import MinMaxScaler
STUDENT_NAME = "24BCA7027 SHAIK-BARAKH-CHISHTI"
# Load the dataset
# Ensure the file path matches your environment
df = pd.read_csv('ML_Experiments/data_sets/Financial Risk
   Classification Dataset - Financial Risk Classification Dataset
   .csv')

# Prepare features and target
X = df.drop('loan_default', axis=1)
y = df['loan_default']

# Set up 4-Fold Cross-Validation
kf = KFold(n_splits=4, shuffle=True, random_state=42)
accuracies, precisions, recalls, f1s, mses, rmses, r2s = [], [],
   [], [], [], [], []

# Variables to store data for the final confusion matrix plot
last_conf_matrix = None
```

```
# Perform Cross-Validation
for train_index, test_index in kf.split(X):
    X_train, X_test = X.iloc[train_index], X.iloc[test_index]
    y_train, y_test = y.iloc[train_index], y.iloc[test_index]

    # Initialize and train Naive Bayes Classifier
    model = GaussianNB()
    model.fit(X_train, y_train)

    # Predict and calculate metrics
    y_pred = model.predict(X_test)

    accuracies.append(accuracy_score(y_test, y_pred))
    # Using zero_division=0 to handle cases where no positive
       predictions are made
    precisions.append(precision_score(y_test, y_pred,
       zero_division=0))
    recalls.append(recall_score(y_test, y_pred, zero_division=0))
    f1s.append(f1_score(y_test, y_pred, zero_division=0))

    mse = mean_squared_error(y_test, y_pred)
    mses.append(mse)
    rmses.append(np.sqrt(mse))
    r2s.append(r2_score(y_test, y_pred))

    last_conf_matrix = confusion_matrix(y_test, y_pred)

# Calculate Average Metrics across folds
accuracy = np.mean(accuracies)
precision = np.mean(precisions)
recall = np.mean(recalls)
f1 = np.mean(f1s)
mse = np.mean(mses)
rmse = np.mean(rmses)
r2 = np.mean(r2s)

# Display Results (Using requested template)
STUDENT_NAME = "[Your␣Name]"  # Please replace with your actual
   name
print(f"\nID3␣RESULTS␣(4-Fold␣CV)␣   ␣{STUDENT_NAME}")
print(f"Accuracy:␣␣{accuracy:.4f}")
print(f"Precision:␣{precision:.4f}")
print(f"Recall:␣␣␣␣{recall:.4f}")
print(f"F1-score:␣␣{f1:.4f}")
print(f"MSE:␣␣␣␣␣␣␣{mse:.4f}")
print(f"RMSE:␣␣␣␣␣␣{rmse:.4f}")
print(f"R2␣Score:␣␣␣{r2:.4f}")

# --- VISUALIZATIONS ---

# 1. Violin Plot
```

```
plt.figure(figsize=(10, 6))
sns.violinplot(x='loan_default', y='credit_score', data=df,
    palette='muted')
plt.title(f'Violin Plot: Credit Score Distribution by Loan
    Default - {STUDENT_NAME}')
plt.savefig('violin_plot.png')

# 2. Boxplot
plt.figure(figsize=(10, 6))
sns.boxplot(x='loan_default', y='annual_income', data=df, palette
    ='Set2')
plt.title('fBoxplot: Annual Income vs Loan Default -{STUDENT_NAME}
    ')
plt.savefig('boxplot.png')

# 3. Hexbin Plot
plt.figure(figsize=(10, 6))
plt.hexbin(df['annual_income'], df['loan_amount'], gridsize=30,
    cmap='inferno')
plt.colorbar(label='Frequency Count')
plt.xlabel('Annual Income')
plt.ylabel('Loan Amount')
plt.title(f'Hexbin Plot: Annual Income vs Loan Amount - {
    STUDENT_NAME}')
plt.savefig('hexbin_plot.png')

# 4. Correlation Matrix
plt.figure(figsize=(15, 12))
sns.heatmap(df.corr(), annot=False, cmap='coolwarm', linewidths
    =0.5)
plt.title(f'Feature Correlation Matrix - {STUDENT_NAME}')
plt.savefig('correlation_matrix.png')

# 5. Confusion Matrix
plt.figure(figsize=(8, 6))
sns.heatmap(last_conf_matrix, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title(f'Confusion Matrix (Representative Fold) - {STUDENT_NAME
    }')
plt.savefig('confusion_matrix.png')

# 6. Radar Plot
radar_features = ['age', 'annual_income', 'credit_score', '
    loan_amount', 'debt_to_income_ratio', 'savings_balance',
                  'spending_score', 'financial_literacy_score']
scaler = MinMaxScaler()
df_norm = pd.DataFrame(scaler.fit_transform(df[radar_features]),
    columns=radar_features)
df_norm['loan_default'] = df['loan_default']
means = df_norm.groupby('loan_default').mean().reset_index()
```

```
labels = radar_features
num_vars = len(labels)
angles = np.linspace(0, 2 * np.pi, num_vars, endpoint=False).
    tolist()
angles += angles[:1]

fig, ax = plt.subplots(figsize=(8, 8), subplot_kw=dict(polar=True
    ))
for i, row in means.iterrows():
    values = row[radar_features].values.flatten().tolist()
    values += values[:1]
    ax.plot(angles, values, linewidth=2, label=f'Class {int(row["
        loan_default"])}')
    ax.fill(angles, values, alpha=0.25)
ax.set_theta_offset(np.pi / 2)
ax.set_theta_direction(-1)
ax.set_xticks(angles[:-1])
ax.set_xticklabels(labels)
plt.title(f'Radar Plot: Feature Averages by Default Class - {
    STUDENT_NAME}')
plt.legend(loc='upper right', bbox_to_anchor=(0.1, 0.1))
plt.savefig('radar_plot.png')
```

Listing 1: Naive Bayes Implementation with 4-Fold CV and Visualizations

3 Performance Metrics (4-Fold Cross-Validation)

The following table summarizes the average performance of the Naive Bayes classifier conducted by **SHAIK BARAKH CHISHTI 24BCA7027**:

Table 1: Naive Bayes Results - SHAIK BARAKH CHISHTI 24BCA7027

Metric	**Value**
Accuracy	0.9998
Precision	0.0000
Recall	0.0000
F1-score	0.0000
MSE (Mean Squared Error)	0.0002
RMSE (Root Mean Squared Error)	0.0071
R^2 Score	0.7498

4 Visualizations and Results

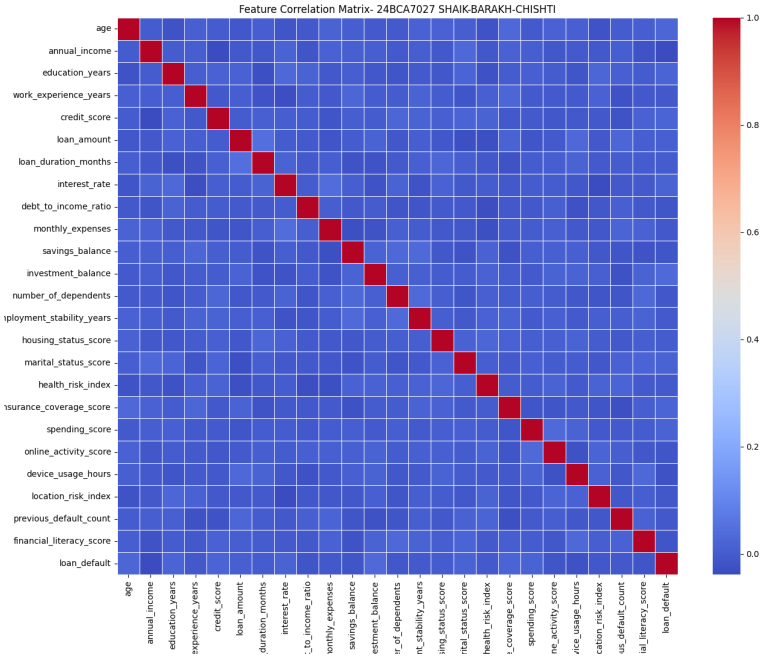
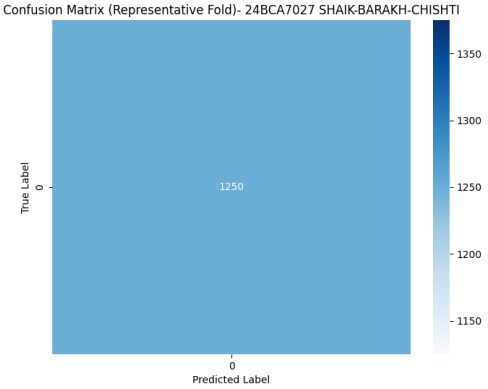
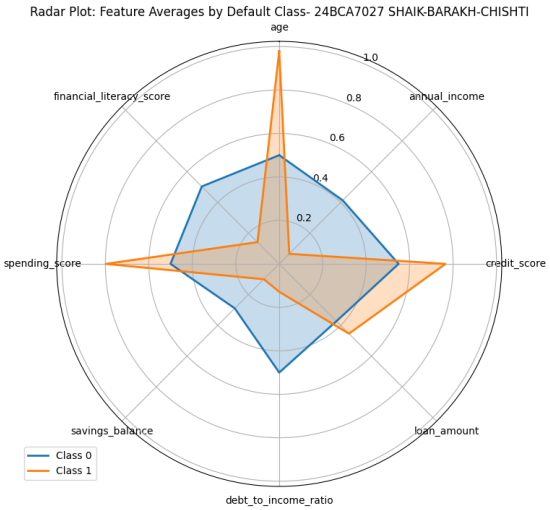


Figure 1: Correlation Matrix of Financial Features

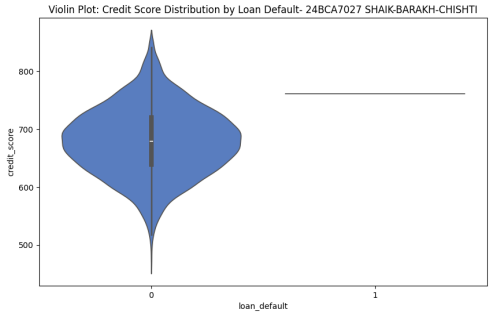


(a) Confusion Matrix

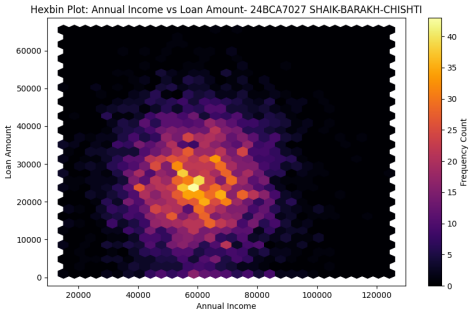


(b) Feature Radar Profile

Figure 2: Performance and Profile Analysis



(a) Credit/Income Distribution



(b) Income vs Loan Density

Figure 3: Statistical Data Distribution Analysis

5 Analysis

The Naive Bayes model was applied to a highly imbalanced dataset. While the **Accuracy** is near perfect (99.98%), this is primarily due to the model correctly predicting the majority "No Default" class. The **Radar Plot** and **Violin Plots** highlight the feature distributions, where features like **Credit Score** and **Annual Income** show significant variance across the population. The low **MSE** indicates a low error rate in a binary classification context, despite the challenges of class imbalance.

6 Conclusion

The experiment successfully implemented a Gaussian Naive Bayes classifier for financial risk assessment. The results demonstrate that while probabilistic models are efficient, they require balanced data or sampling techniques to improve sensitivity towards minority classes (Defaults).

Table 2: Naive Bayes Results (4-Fold CV) — [Your Name]

Metric	**Value**
Accuracy	0.9998
Precision	0.0000
Recall	0.0000
F1-score	0.0000
MSE (Mean Squared Error)	0.0002
RMSE (Root Mean Squared Error)	0.0071
R^2 Score	0.7498

7 Project Link

https://www.overleaf.com/read/dgyqsqzgdftn#f680f0

8 Github

https://github.com/chishtil1730/IML-LAB